Bash { SNBV™ }

1 message

THE KING ROYAL MAYOR FIELD MARSHAL <theking@kingdomofmurabba.org> Tue, Oct 20, 2026 at 00:20
To: A00455717@exatec.tec.mx, SERGIO Berlanga <sergiob@prodigy.net.mx>, SERGIO Berlanga <sergio_b_@hotmail.com>, CHEK0 Berlanga <sergioberlangav@gmail.com>, theking@kingdomofmurabba.org, sergio_berlanga@colpal.com

Bash

{

Absolutely — here is the final full single-repo code bundle, fully wired together.

This is the complete app shell for:
- React + Vite + TypeScript
- Zustand state
- typed Event Bus
- Three.js Globe
- Radar Canvas
- Campaign Panel
- Telemetry stream bridge
- Electron desktop packaging

Copy these files into a project root and run:
- npm install
- npm run build
- npm run electron:dev

1) package.json

```json
{
  "name": "murabba-command-core",
  "version": "1.0.0",
  "private": true,
  "main": "electron/main.js",
  "scripts": {
    "dev": "vite",
    "build": "tsc && vite build",
    "preview": "vite preview",
    "electron:dev": "cross-env NODE_ENV=development concurrently \"vite\" \"wait-on http://localhost:5173 && electron .\"",
    "electron:pack": "npm run build && electron-builder",
    "electron:dist": "npm run build && electron-builder --publish=never"
  },
  "dependencies": {
    "@react-three/drei": "^9.0.0",
    "@react-three/fiber": "^8.0.0",
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "three": "^0.165.0",
    "zustand": "^4.5.0"
  },
  "devDependencies": {
    "@types/react": "^18.2.0",
    "@types/react-dom": "^18.2.0",
    "@vitejs/plugin-react": "^4.3.0",
    "concurrently": "^8.2.0",
    "cross-env": "^7.0.0",
    "electron": "^28.0.0",
    "electron-builder": "^24.9.0",
    "typescript": "^5.4.0",
    "vite": "^5.4.0",
    "wait-on": "^7.0.0"
  },
  "build": {
    "appId": "com.murabba.commandcore",
    "productName": "Murabba Command Core",
    "files": [
      "dist/**/*",
      "electron/**/*",
      "package.json"
    ],
    "directories": {
      "output": "release"
```

```
    },
    "win": {
      "target": "nsis",
      "artifactName": "${productName}-${version}-setup.${ext}"
    },
    "mac": {
      "target": "dmg",
      "artifactName": "${productName}-${version}.${ext}"
    },
    "linux": {
      "target": "AppImage",
      "artifactName": "${productName}-${version}.${ext}"
    }
  }
}
```

2) vite.config.ts

```ts
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import path from "path";

export default defineConfig({
  plugins: [react()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
});
```

3) tsconfig.json

```json
{
  "compilerOptions": {
    "target": "ES2020",
    "useDefineForClassFields": true,
    "lib": ["DOM", "DOM.Iterable", "ES2020"],
    "allowJs": false,
    "skipLibCheck": true,
    "esModuleInterop": true,
    "allowSyntheticDefaultImports": true,
    "strict": true,
    "forceConsistentCasingInFileNames": true,
    "module": "ESNext",
    "moduleResolution": "Node",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "baseUrl": ".",
    "paths": {
      "@/*": ["src/*"]
    }
  },
  "include": ["src", "electron"]
}
```

4) index.html

```html
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Murabba Command Core</title>
    <meta name="theme-color" content="#07090e" />
  </head>
  <body>
    <div id="root"></div>
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

5) electron/main.js

```js
const { app, BrowserWindow } = require("electron");
const path = require("path");

function createWindow() {
  const mainWindow = new BrowserWindow({
    width: 1440,
    height: 900,
    minWidth: 1200,
    minHeight: 760,
    backgroundColor: "#07090e",
    titleBarStyle: "hiddenInset",
    autoHideMenuBar: true,
    webPreferences: {
      preload: path.join(__dirname, "preload.js"),
      nodeIntegration: false,
      contextIsolation: true,
      sandbox: false,
    },
  });

  const isDev = process.env.NODE_ENV === "development" || !app.isPackaged;

  if (isDev) {
    mainWindow.loadURL("http://localhost:5173");
    mainWindow.webContents.openDevTools();
  } else {
    mainWindow.loadFile(path.join(__dirname, "../dist/index.html"));
  }
}

app.whenReady().then(() => {
  createWindow();

  app.on("activate", () => {
    if (BrowserWindow.getAllWindows().length === 0) {
      createWindow();
    }
  });
});

app.on("window-all-closed", () => {
  if (process.platform !== "darwin") {
    app.quit();
  }
});
```

6) electron/preload.js

```js
const { contextBridge } = require("electron");

contextBridge.exposeInMainWorld("appMeta", {
  name: "Murabba Command Core",
  version: process.env.npm_package_version || "1.0.0",
  platform: process.platform,
});
```

7) src/main.tsx

```tsx
import React from "react";
import ReactDOM from "react-dom/client";
import { AppRoot } from "./AppRoot";
import "./styles/index.css";

ReactDOM.createRoot(document.getElementById("root")!).render(
  <React.StrictMode>
    <AppRoot />
  </React.StrictMode>
);
```

8) src/styles/index.css

```css
:root {
```

```
  --bg-primary: #07090e;
  --bg-secondary: rgba(12, 17, 24, 0.9);
  --bg-panel: rgba(12, 17, 24, 0.75);
  --border-primary: rgba(255, 255, 255, 0.08);
  --text-primary: #edf6ff;
  --text-muted: #9ab0c5;
  --accent-primary: #ffd978;
  --accent-bright: #00f0ff;
  --accent-cyan: #00f0ff;
  --accent-gold: #ffd978;
  --danger: #ff5f57;
  --success: #57ff85;
  --warning: #ffcc66;
}

* {
  box-sizing: border-box;
}

html,
body,
#root {
  margin: 0;
  width: 100%;
  min-height: 100%;
  background: var(--bg-primary);
  color: var(--text-primary);
  font-family: "JetBrains Mono", "Segoe UI", sans-serif;
}

body {
  min-height: 100vh;
  background:
    radial-gradient(circle at top, rgba(0, 240, 255, 0.06), transparent 25%),
    radial-gradient(circle at bottom, rgba(255, 217, 120, 0.06), transparent 25%),
    var(--bg-primary);
}

button,
input {
  font: inherit;
}

button {
  cursor: pointer;
}

a {
  color: inherit;
  text-decoration: none;
}
```

9) src/store/store.types.ts

```ts
export type CoreStatus = "nominal" | "warning" | "critical";
export type ThemePreset = "sovereign-gold" | "midnight-cyan" | "ember-rose";
export type TelemetryLevel = "info" | "warning" | "critical";

export interface TelemetryEvent {
  id: string;
  timestamp: string;
  category: string;
  message: string;
  level: TelemetryLevel;
}

export interface AppState {
  selectedNodeId: string | null;
  setSelectedNodeId: (id: string | null) => void;

  coreStatus: CoreStatus;
  setCoreStatus: (status: CoreStatus) => void;

  logs: TelemetryEvent[];
  addLog: (category: string, message: string, level?: TelemetryLevel) => void;
  clearLogs: () => void;

  theme: ThemePreset;
  setTheme: (theme: ThemePreset) => void;
```

```
  crtFilterEnabled: boolean;
  toggleCrtFilter: () => void;

  audioEnabled: boolean;
  toggleAudio: () => void;
}
```

10) src/store/useAppStore.ts

```ts
import { create } from "zustand";
import { persist, createJSONStorage } from "zustand/middleware";
import { WORLD_NODES } from "@/features/world/worldUtils";
import type { AppState, CoreStatus, ThemePreset, TelemetryEvent, TelemetryLevel } from "./store.types";

export const useAppStore = create<AppState>()(
  persist(
    (set, get) => ({
      selectedNodeId: "riyadh",

      setSelectedNodeId: (id: string | null) => {
        set({ selectedNodeId: id });

        if (!id) return;

        const resolvedNode = WORLD_NODES.find((node) => node.id === id);
        if (!resolvedNode) return;

        get().addLog(
          "RADAR",
          `Target locked: ${resolvedNode.name} [${resolvedNode.classification}]`,
          resolvedNode.status === "warning" ? "warning" : "info"
        );
      },

      coreStatus: "nominal",
      setCoreStatus: (status: CoreStatus) => {
        set({ coreStatus: status });
        get().addLog("MUKAAB", `Core status changed: ${status.toUpperCase()}`, status === "warning" ? "warning" : "info");
      },

      logs: [
        {
          id: crypto.randomUUID(),
          timestamp: new Date().toLocaleTimeString(),
          category: "SYSTEM",
          message: "KINGDOM OF MURABBA™ Command Core online. Golden Ratio Φ active.",
          level: "info",
        },
      ],

      addLog: (category: string, message: string, level: TelemetryLevel = "info") => {
        const event: TelemetryEvent = {
          id: crypto.randomUUID(),
          timestamp: new Date().toLocaleTimeString(),
          category,
          message,
          level,
        };

        set((state) => ({
          logs: [event, ...state.logs].slice(0, 100),
        }));
      },

      clearLogs: () => set({ logs: [] }),

      theme: "sovereign-gold",
      setTheme: (theme: ThemePreset) => {
        set({ theme });
        get().addLog("SYSTEM", `Visual theme updated: ${theme}`, "info");
      },

      crtFilterEnabled: true,
      toggleCrtFilter: () => set((state) => ({ crtFilterEnabled: !state.crtFilterEnabled })),

      audioEnabled: true,
      toggleAudio: () => set((state) => ({ audioEnabled: !state.audioEnabled })),
    }),
```

```
    {
      name: "snvb-command-core-storage",
      storage: createJSONStorage(() => localStorage),
      partialize: (state) => ({
        theme: state.theme,
        crtFilterEnabled: state.crtFilterEnabled,
        audioEnabled: state.audioEnabled,
        selectedNodeId: state.selectedNodeId,
      }),
    }
  )
);
```

11) src/systems/eventBus/eventBus.types.ts

```ts
import type { CoreStatus, ThemePreset } from "@/store/store.types";

export interface NodeEventPayload {
  id: string;
  source: "radar" | "globe" | "campaign" | "commandPalette";
  timestamp?: number;
}

export interface EventMap {
  "radar:node:selected": NodeEventPayload;
  "radar:node:hovered": { id: string | null; source: string };
  "radar:pulse:executed": { timestamp: number; origin: string };

  "world:node:selected": NodeEventPayload;
  "world:camera:focus": { latitude: number; longitude: number; name: string };

  "mukaab:status:changed": { previous: CoreStatus; current: CoreStatus };
  "mukaab:core:clicked": { timestamp: number };

  "system:theme:changed": { theme: ThemePreset };
  "system:crt:toggled": { enabled: boolean };
  "system:audio:toggled": { enabled: boolean };

  "telemetry:log:added": { category: string; message: string; level: string };
  "telemetry:logs:cleared": { timestamp: number };
  "command:executed": { commandId: string; title: string };
}
```

12) src/systems/eventBus/eventBus.ts

```ts
import type { EventMap } from "./eventBus.types";

type EventName = keyof EventMap;
type Listener<K extends EventName> = (payload: EventMap[K]) => void;

class EventBus {
  private listeners = new Map<EventName, Set<Function>>();

  public on<K extends EventName>(event: K, listener: Listener<K>) {
    if (!this.listeners.has(event)) {
      this.listeners.set(event, new Set());
    }

    this.listeners.get(event)!.add(listener as Function);
    return () => this.off(event, listener);
  }

  public off<K extends EventName>(event: K, listener: Listener<K>) {
    const set = this.listeners.get(event);
    if (!set) return;
    set.delete(listener as Function);
  }

  public emit<K extends EventName>(event: K, payload: EventMap[K]) {
    const set = this.listeners.get(event);
    if (!set) return;

    for (const listener of set.values()) {
      listener(payload);
    }
  }
}
```

```
export const eventBus = new EventBus();
```

13) src/features/world/world.types.ts

```ts
export interface WorldNode {
  id: string;
  name: string;
  latitude: number;
  longitude: number;
  color: string;
  category: "command" | "observer" | "relay" | "telemetry";
  status: "active" | "warning" | "critical";
  classification: string;
  description?: string;
}
```

14) src/features/world/world.constants.ts

```ts
import type { WorldNode } from "./world.types";

export const GLOBE_RADIUS = 2.0;

export const WORLD_NODES: WorldNode[] = [
 {
   id: "riyadh",
   name: "RIYADH CORE",
   latitude: 24.7136,
   longitude: 46.6753,
   color: "#ffd978",
   category: "command",
   status: "active",
   classification: "Laser Core",
   description: "Primary sovereign command node",
 },
 {
   id: "cairo",
   name: "CAIRO RELAY",
   latitude: 30.0444,
   longitude: 31.2357,
   color: "#dfb15b",
   category: "relay",
   status: "active",
   classification: "Resonance Array",
   description: "Regional relay and signal mesh",
 },
 {
   id: "london",
   name: "LONDON OBSERVER",
   latitude: 51.5074,
   longitude: -0.1278,
   color: "#00f0ff",
   category: "observer",
   status: "active",
   classification: "Naval Telemetry",
   description: "Maritime and orbital observation layer",
 },
 {
   id: "vatican",
   name: "VATICAN ARRAY",
   latitude: 41.9029,
   longitude: 12.4534,
   color: "#ffe600",
   category: "telemetry",
   status: "warning",
   classification: "Observer Node",
   description: "Deep acoustic observatory",
 },
];
```

15) src/features/world/worldUtils.ts

```ts
import * as THREE from "three";
import type { WorldNode } from "./world.types";
import { WORLD_NODES as WORLD_NODES_CONST } from "./world.constants";
```

```
export const GLOBE_RADIUS = 2.0;
export const WORLD_NODES: WorldNode[] = WORLD_NODES_CONST;

export function latLngToVector3(lat: number, lng: number, radius: number = GLOBE_RADIUS): THREE.Vector3 {
  const phi = (90 - lat) * (Math.PI / 180);
  const theta = (lng + 180) * (Math.PI / 180);

  const x = -(radius * Math.sin(phi) * Math.cos(theta));
  const z = radius * Math.sin(phi) * Math.sin(theta);
  const y = radius * Math.cos(phi);

  return new THREE.Vector3(x, y, z);
}

export function getWorldNodeById(id: string | null): WorldNode | null {
  if (!id) return null;
  return WORLD_NODES.find((node) => node.id === id) ?? null;
}
```

16) src/features/world/GlobeCameraController.ts

```ts
import { useEffect } from "react";
import { useThree } from "@react-three/fiber";
import { eventBus } from "@/systems/eventBus";
import { WORLD_NODES, latLngToVector3, GLOBE_RADIUS } from "./worldUtils";

export function GlobeCameraController() {
  const { camera } = useThree();

  useEffect(() => {
    const unsub = eventBus.on("radar:node:selected", ({ id }) => {
      const worldNode = WORLD_NODES.find((node) => node.id === id);
      if (!worldNode) return;

      const target = latLngToVector3(worldNode.latitude, worldNode.longitude, GLOBE_RADIUS + 3.2);

      const start = camera.position.clone();
      let progress = 0;

      const animate = () => {
        progress += 0.04;
        camera.position.lerpVectors(start, target, Math.min(progress, 1));
        camera.lookAt(0, 0, 0);

        if (progress < 1) {
          requestAnimationFrame(animate);
        }
      };

      animate();
    });

    return () => unsub();
  }, [camera]);

  return null;
}
```

17) src/features/world/GlobeMarkers.tsx

```tsx
import React, { useRef } from "react";
import { useFrame } from "@react-three/fiber";
import * as THREE from "three";
import { WORLD_NODES, latLngToVector3, GLOBE_RADIUS } from "./worldUtils";
import { eventBus } from "@/systems/eventBus";
import { useAppStore } from "@/store/useAppStore";
import type { WorldNode } from "./world.types";

interface Props {
  selectedNodeId: string | null;
}

export const GlobeMarkers: React.FC<Props> = ({ selectedNodeId }) => {
  return (
    <group>
      {WORLD_NODES.map((node) => (
```

```
      <MarkerItem key={node.id} node={node} isSelected={selectedNodeId === node.id} />
    ))}
  </group>
 );
};

const MarkerItem: React.FC<{ node: WorldNode; isSelected: boolean }> = ({ node, isSelected }) => {
 const ringRef = useRef<THREE.Mesh>(null);
 const setSelectedNodeId = useAppStore((state) => state.setSelectedNodeId);

 const pos = latLngToVector3(node.latitude, node.longitude, GLOBE_RADIUS + 0.02);

 useFrame((state) => {
  if (ringRef.current) {
   const time = state.clock.getElapsedTime();
   const scale = 1 + Math.sin(time * 3) * 0.25;
   ringRef.current.scale.set(scale, scale, scale);
  }
 });

 return (
  <group position={pos}>
   <mesh
    onClick={(e) => {
     e.stopPropagation();
     setSelectedNodeId(node.id);
     eventBus.emit("radar:node:selected", {
      id: node.id,
      source: "globe",
      timestamp: Date.now(),
     });
    }}
   >
    <sphereGeometry args={[0.035, 16, 16]} />
    <meshBasicMaterial color={isSelected ? "#00f0ff" : node.color} />
   </mesh>

   <mesh ref={ringRef}>
    <ringGeometry args={[0.045, 0.055, 32]} />
    <meshBasicMaterial
     color={isSelected ? "#00f0ff" : node.color}
     side={THREE.DoubleSide}
     transparent
     opacity={0.6}
    />
   </mesh>
  </group>
 );
};
```

18) src/features/world/GlobeScene.tsx

```tsx
import React, { useRef } from "react";
import { useFrame } from "@react-three/fiber";
import * as THREE from "three";
import { GlobeMarkers } from "./GlobeMarkers";
import { GlobeCameraController } from "./GlobeCameraController";
import { GLOBE_RADIUS } from "./worldUtils";

interface Props {
 selectedNodeId: string | null;
}

export const GlobeScene: React.FC<Props> = ({ selectedNodeId }) => {
 const globeRef = useRef<THREE.Group>(null);

 useFrame((_, delta) => {
  if (globeRef.current && !selectedNodeId) {
   globeRef.current.rotation.y += delta * 0.05;
  }
 });

 return (
  <group ref={globeRef}>
   <GlobeCameraController />

   <mesh>
    <sphereGeometry args={[GLOBE_RADIUS, 64, 64]} />
    <meshStandardMaterial color="#07090e" metalness={0.9} roughness={0.2} />
```

```
      </mesh>

      <mesh>
        <sphereGeometry args={[GLOBE_RADIUS + 0.005, 36, 36]} />
        <meshBasicMaterial color="#dfb15b" wireframe transparent opacity={0.12} />
      </mesh>

      <mesh>
        <sphereGeometry args={[GLOBE_RADIUS + 0.15, 32, 32]} />
        <meshBasicMaterial color="#00f0ff" transparent opacity={0.05} side={THREE.BackSide} />
      </mesh>

      <GlobeMarkers selectedNodeId={selectedNodeId} />
    </group>
  );
};
```

19) src/features/world/WorldGlobe.tsx

```tsx
import React, { Suspense, useEffect } from "react";
import { Canvas } from "@react-three/fiber";
import { OrbitControls } from "@react-three/drei";
import { GlobeScene } from "./GlobeScene";
import { useAppStore } from "@/store/useAppStore";
import { WORLD_NODES } from "./worldUtils";
import { eventBus } from "@/systems/eventBus";
import styles from "./world.module.css";

export default function WorldGlobe() {
  const selectedNodeId = useAppStore((state) => state.selectedNodeId);
  const setSelectedNodeId = useAppStore((state) => state.setSelectedNodeId);

  const selectedNode = WORLD_NODES.find((node) => node.id === selectedNodeId) ?? null;

  useEffect(() => {
    const unsub = eventBus.on("radar:node:selected", ({ id, source }) => {
      if (source !== "globe") {
        setSelectedNodeId(id);
      }
    });

    return () => unsub();
  }, [setSelectedNodeId]);

  return (
    <div className={styles.globeContainer}>
      <div className={styles.header}>
        <span className={styles.title}>SOVEREIGN 3D GLOBE TELEMETRY</span>
        <span className={styles.coords}>{selectedNode ? selectedNode.name : "GLOBAL ORBIT"}</span>
      </div>

      <Canvas camera={{ position: [0, 0, 5.5], fov: 45 }} gl={{ antialias: true, alpha: true }} className={styles.canvas}>
        <ambientLight intensity={0.5} />
        <directionalLight position={[5, 3, 5]} intensity={1.2} color="#ffffff" />
        <pointLight position={[-5, -3, -5]} intensity={0.8} color="#00f0ff" />

        <Suspense fallback={null}>
          <GlobeScene selectedNodeId={selectedNodeId} />
        </Suspense>

        <OrbitControls enableZoom maxDistance={8} minDistance={3.5} enablePan={false} rotateSpeed={0.5} />
      </Canvas>
    </div>
  );
}
```

20) src/features/world/world.module.css

```css
.globeContainer {
  position: relative;
  width: 100%;
  height: 100%;
  min-height: 420px;
  background: rgba(7, 9, 14, 0.95);
  border: 1px solid var(--border-primary);
  border-radius: 8px;
  box-shadow: 0 8px 32px
```

}

https://www.linkedin.com/in/MURABBAKING
https://www.kingdomofmurabba.org
ملك المرابّا سيرجيو نيكولا بيرلانغا فياريال

رئيس البلدية الملكي، المشير